

PRESENTATION & VIVA GUIDE

Chronos at Its Current Stage

One-minute pitch, architecture, demo script, cross-questions, evidence, and honest limitations

CURRENT REMOTE STAGE | origin/main f1e11b4 | 18 August 2026

In-memory research prototype - not yet a production database

Best one-line description: Chronos is a Git-like versioning layer for key-value data that uses content-addressed hash trees to share unchanged structure, create historical versions, skip identical subtrees during diff, and reconstruct older states.

Prepared for a teacher review, live demonstration, and technical viva

1. The one-minute pitch

Section focus: Memorise the structure, not every word. Deliver it slowly enough that four proof points land: versions, sharing, diff pruning, and measured evidence.

Chronos is a Git-like versioning layer for data. Traditional state snapshots copy the whole dataset for every version, while operation logs keep only changes but must replay history to reconstruct a state. Chronos combines useful parts of both approaches. It converts records into canonical key-value entries and stores them in a content-addressed SHA-256 tree, so identical nodes have identical hashes and are stored only once. A change copies only the affected path in the fixed hash trie; unchanged branches are shared across versions. Each commit records a root hash, parent, message, timestamp, and version. During diff, equal subtree hashes prove that those regions are identical, so Chronos skips them instead of scanning every key. We now have incremental writes, commit history, checkout, and two engines: a fixed hash trie and an ordered Prolly-tree prototype. All 38 tests pass. In our 100,000-key benchmark, three updates created only 15 trie nodes and the incremental commit took about 0.8 milliseconds, compared with about 883 milliseconds for a full rebuild.

Pitch beat	What the listener should remember
Problem	Full snapshots duplicate data; logs make old-state reconstruction depend on replay.
Mechanism	Content-addressed hash trees give each version a root while sharing unchanged nodes.
Core proof	Matching subtree hashes let diff stop early; path copying limits a trie update to the changed route.
Evidence	38 tests pass; 3 updates at 100K keys created 15 nodes and committed in about 0.8 ms here.
Boundary	This is an in-memory research prototype; durability, recovery, branching, and merge remain future work.

Closing sentence: Chronos proves that versioning can preserve complete states without copying every state, while diff work follows changed structure rather than total dataset size.

2. Our project description

Section focus: Use this section when your teacher asks, 'What exactly have you built?' or 'Is this state-based or operational?'

Problem: Versioned systems usually choose between complete snapshots, which are simple but duplicate unchanged data, and operation logs, which store changes cheaply but can require replay and longer-history reasoning.

Our solution: Chronos stores immutable nodes by the SHA-256 hash of canonical content. A version is represented by a root hash, and roots can share unchanged descendants.

What a commit means: A commit is metadata that points to a state root. It records version number, root hash, parent version, message, and timestamp; HEAD points to the latest recorded commit.

Why it is both state and operational: The root hash represents the complete state, while put/delete operations and SQL-history text describe how the state changed. Chronos does not replay SQL text to prove or reconstruct the committed state.

Current capabilities

- Initial full commit from a Python mapping and incremental put/delete updates from an existing root.
- Linear commit graph with HEAD, commit messages, timestamps, `log()`, `checkout()`, and `materialize()`.
- Hash-pruned diff that returns changed keys and reports compared node pairs and skipped matching subtrees.
- Fixed SHA-256 trie for fast point updates and an ordered Prolly-tree prototype for stable chunks and efficient range scans.
- CSV snapshot adapter and SQL-history metadata for a MySQL-export demonstration workflow.
- Benchmarks covering commit latency, diff latency/work, storage or node creation, scaling, and range scans.

Correct stage label: An in-memory versioned-data research prototype with working incremental history, checkout, diff, and two experimental content-addressed engines. It is not yet a durable DBMS or a MySQL storage engine.

3. Architecture and data flow

Section focus: Explain the system from input to version root, then show how a later diff uses the stored hashes.

Layer	Current responsibility	Key objects / APIs
Input / adapter	Reads mappings or exported CSV and normalises rows into stable key-value records. SQL history is metadata.	csv_snapshot_demo.py
Versioning API	Creates full commits or applies put/delete to a root; records commit metadata and HEAD.	commit, put, delete, log, checkout
Hash engine	Routes or chunks keys, canonicalises nodes, hashes content, and performs immutable path copying.	VersionedDatabase / ProollyStore
Node store	Dictionary keyed by node hash. The same hash refers to the same immutable node.	node_store[hash]
Version store	Maps versions to roots and stores a linear parent chain with messages and timestamps.	versions, commits, head
Comparison	Walks both roots; equal hashes stop recursion, unequal leaves identify changed keys.	diff(root_a, root_b)

The fixed hash-trie path

1. Hash the logical key with SHA-256 and use successive hash portions to choose one of eight child slots at each level.
2. At the leaf, store the canonical key-value entry and calculate the leaf hash.
3. For put/delete, create the changed leaf and copy only its four ancestors; every unaffected sibling hash is reused.
4. The new top hash becomes the state root. A single-key update creates at most five trie nodes in the current depth-four design.

The Proolly-tree path

- Sort keys and create content-defined chunks using deterministic boundaries; the target is about 32 entries per leaf.
- Because order is preserved, range_scan() can seek and return neighbouring keys efficiently.
- The root is history-independent for the same ordered state, but current internal levels are rebuilt from the leaf list after an update, so writes are not yet $O(\text{depth})$.

4. How to demonstrate it live

Section focus: Run commands separately and narrate one claim per command. These commands describe the updated remote stage after it is safely integrated locally.

Important before the demo: Your local main branch is six commits behind origin/main and has local edits in README.md, csv_snapshot_demo.py, and test_csv_snapshot_demo.py. Commit or stash those edits and integrate carefully; do not pull blindly because two areas overlap conceptually.

Recommended command sequence

```
python -m unittest -v
python incremental_demo.py
python incremental_benchmark.py
python compare_engines.py
```

Command	What it proves	What to say
unittest	Correctness across 38 tests	The benchmark is accepted only after core and engine behaviours pass.
incremental_demo	put/delete, commit chain, log, checkout, diff	An old root remains unchanged; HEAD points to the new state; diff returns exactly the modified keys.
incremental_benchmark	Update work follows changed paths	At 100K keys, three changes create 15 trie nodes instead of rebuilding the whole tree.
compare_engines	Trie / Prolly workload trade-off	Trie wins current writes; Prolly wins ordered range scan. Neither is universally better.

Thirty-second narration for the incremental demo

We load 20,000 records, create an initial root, then update, insert, and delete individual keys. Every new commit points to a new root, but unchanged nodes remain shared. `log()` shows the parent chain, `checkout()` materialises an older version without changing it, and `diff()` finds only `users:1`, `users:500`, and `users:999999` while skipping matching subtrees.

5. Numbers to remember

Section focus: Use these as evidence, not universal performance promises. Timings are from the current Windows/Python verification environment and will vary by machine.

Evidence	Verified result	Meaning
Automated tests	38 / 38 passed	Core trie, Prolly engine, commits, checkout, diff, and benchmarks behaved as expected.
20K-key demo	1 put = 5 nodes; 0.207 ms	The fixed trie copies one leaf path rather than rebuilding the state.
100K, 3 changed keys	Full 882.804 ms; incremental 0.786 ms	About 1,123x faster for this workload; 15 new nodes.
Scale at 100K	957.422 ms vs 2.355 ms	Full rebuild versus incremental path update: about 406.5x.
Ordered range scan	Trie 50.90 ms; Prolly 0.47 ms	Prolly was about 109.2x faster for a 1,000-key scan.
Bulk append: 200 at 100K	Trie 38.7 ms; Prolly 2,579.2 ms	Current Prolly rebuilding made writes about 66.6x slower.
Diff: 5 edits	Trie 0.22 ms; Prolly 0.52 ms	Both prune by hashes; the fixed trie was faster in this run.

Safest benchmark sentence: The results support a workload-dependent design: the fixed trie currently excels at point updates, while the Prolly engine provides ordered chunks and much faster range scans.

6. Cross-questions - fundamentals

Section focus: Answer the first sentence directly; add the deeper explanation only if asked.

1. What problem does Chronos solve?

Answer: It keeps many dataset versions without copying every unchanged record for every version.

If asked deeper: Immutable content-addressed nodes let roots share descendants, and hashes avoid reading identical regions during diff.

2. Is Chronos a database?

Answer: It is currently an in-memory versioning engine and proof of concept, not a complete production DBMS.

If asked deeper: Production still requires durability, transactions, concurrency control, crash recovery, access control, and tooling.

3. Merkle tree or fixed tree?

Answer: Both: the main engine is a fixed-depth, fixed-fanout hash trie with Merkle-tree properties.

If asked deeper: Fixed describes shape and routing; Merkle describes parents hashing child hashes and content.

4. On what basis is the tree made?

Answer: The trie hashes each key with SHA-256 and uses successive hash portions to select one of eight child slots.

If asked deeper: Prolly instead sorts keys and uses content-defined boundaries for stable ordered chunks.

5. What exactly is diff?

Answer: It compares two roots and returns keys that were added, deleted, or changed.

If asked deeper: Equal hashes prove whole subtrees identical, so recursion stops; only unequal branches are opened.

7. Cross-questions - versions and models

6. How does one changed key create a new version?

Answer: Chronos writes a new leaf and ancestors along that key's route, then records the new root.

If asked deeper: Old nodes remain immutable and sibling hashes are shared by both versions.

7. Are shared versions independent?

Answer: Yes, logically. They share only immutable nodes, so changing a later version cannot alter an older root.

If asked deeper: This is persistent data-structure semantics: copy the changed path, reuse the rest.

8. What commit metadata exists?

Answer: Version, root hash, parent, message, timestamp, and HEAD.

If asked deeper: Metadata is recorded but not yet hashed into a separate tamper-evident commit identity.

9. How does checkout work?

Answer: checkout(version) resolves its root and materialises that historical key-value state without deleting later versions.

If asked deeper: History is currently linear; named branches and merge are not implemented.

10. State-based, operational, or both?

Answer: Both: the root commits to complete state, while put/delete and optional SQL history describe operations.

If asked deeper: Unlike a pure operation log, the state does not require full-history replay.

8. Cross-questions - engines and integration

11. Why have two engines?

Answer: The fixed trie gives fast predictable point updates; Prolly preserves order and supports efficient range scans.

If asked deeper: Engine choice can follow workload rather than claiming a universal winner.

12. Is Prolly better?

Answer: Not universally. It won the ordered range scan but was much slower on current bulk writes.

If asked deeper: Leaf boundaries are incremental, but internal levels are rebuilt; local parent ascent is the next optimisation.

13. Why can an operation log beat Chronos?

Answer: A short log between nearby versions can be extremely cheap to read.

If asked deeper: Its work can grow with history distance and replay; a Chronos root directly represents complete state.

14. How would MySQL connect?

Answer: Use an adapter or transactional outbox that turns committed row events into canonical put/delete operations and one Chronos commit.

If asked deeper: The CSV dry run keeps SQL text as metadata; it does not execute arbitrary SQL.

15. What about collisions and serialisation?

Answer: SHA-256 makes accidental collisions negligible here, and nodes hash canonical serialised content.

If asked deeper: Production should version encodings, verify hashes on read, frame types/lengths, and define corruption handling.

9. What not to overclaim

Section focus: Credibility improves when you state exactly where the proof ends.

Avoid saying	Say this instead
Chronos is faster than operational models.	Trade-offs differ. Logs can win short histories; Chronos gives direct state roots, sharing, and hash-pruned diff.
We built a production database.	We built an in-memory versioning/storage prototype with tests and benchmarks.
Prolly is better than tries.	Prolly improves order/range scans; the current trie writes much faster.
Every update is $O(\text{depth})$.	Fixed-trie updates copy one path; current Prolly updates rebuild internal levels from leaves.
Commit metadata is tamper-evident.	State roots are content-addressed; commit metadata is not yet a separate hashed object.
The MySQL pipeline is complete.	The adapter design and CSV dry run exist; live outbox integration is future work.
Numbers are universal.	These are prototype measurements for a stated dataset, machine, runtime, and workload.

Current limitations

- Nodes and commits are in process memory; restart loses the version store.
- No WAL, durable pages, transactions, concurrency control, crash recovery, authentication, branching, or merge.
- History is linear; recording from an old root still links through current HEAD in the present implementation.
- Prolly proves ordered access and correctness, but its internal update algorithm needs optimisation.
- Production conclusions require disk I/O, memory, concurrency, recovery, and larger real datasets.

10. Presentation checklist and next steps

Section focus: Use this page five minutes before presenting.

Before opening the terminal

- Integrate the six remote commits without overwriting local CSV-adapter edits; run all tests after resolving overlaps.
- Use a large terminal font and run each command separately so one concept appears at a time.
- Keep a screenshot of expected demo output in case the live terminal fails.
- State dataset size and workload before quoting a speed-up; round numbers while speaking.
- Lead with the trie proof, then present Prolly as an ordered experimental alternative.

Recommended next engineering steps

1. Persist nodes and commits behind a storage interface; add WAL, atomic root publication, and restart recovery.
2. Optimise Prolly updates with local parent ascent so changed chunks do not rebuild all internal levels.
3. Hash commit metadata into a true commit identity, then add named branches and three-way merge.
4. Build a MySQL transactional-outbox adapter with idempotency, offsets, retries, schema versioning, and commit grouping.
5. Evaluate real datasets using disk bytes, write amplification, memory, history distance, concurrency, and recovery time.

Final line to your teacher: The current prototype does not claim to replace MySQL; it demonstrates the core storage idea: immutable content-addressed structure can preserve full historical states, reuse unchanged data, and make comparison proportional to changed regions.